

Providing data-driven suggestions for HR

Description

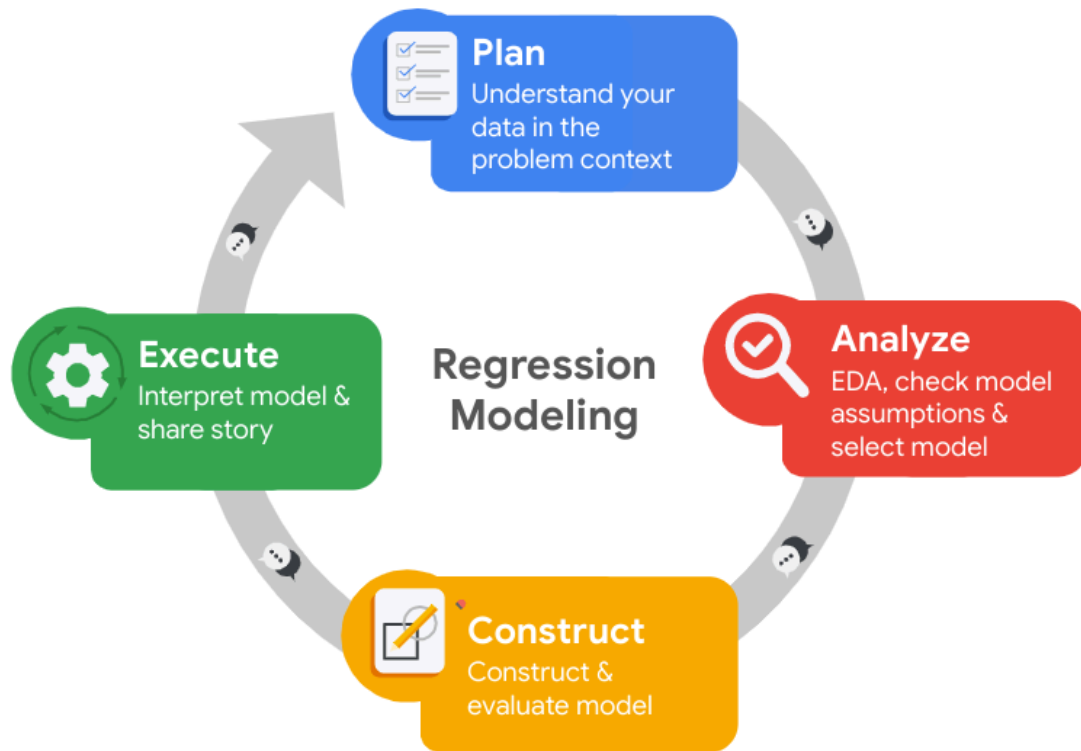
This project is an opportunity to analyse a dataset and construct a model that'll provide insight for the HR department of a big consulting firm.

Together with this project, there'll be an executive summary detailing the key insights acquired during this analysis, that would be presented to stakeholders from Sailfort Motors. This document is the jupyter notebook that contains the complete code used for the analysis.

Objectives:

- 1. Answer the question made by HR: *What's likely to make the employee leave the company?*
- 2. Clean the data according to the necessities of the analysis
- 3. Create visualizations to aid in the interpretation of the relationships between the data.
- 4. Construct a model and evaluate its performance.
- 5. Respect the ethics considerations and data privacy.

PACE stages



Pace: Plan

Consider the questions in your PACE Strategy Document to reflect on the Plan stage.

In this stage, consider the following:

Understand the business scenario and problem

The HR department at Salifort Motors wants to take some initiatives to improve employee satisfaction levels at the company. They collected data from employees, but now they don't know what to do with it. They refer to you as a data analytics professional and ask you to provide data-driven suggestions based on your understanding of the data. They have the following question: what's likely to make the employee leave the company?

Your goals in this project are to analyze the data collected by the HR department and to build a model that predicts whether or not an employee will leave the company.

If you can predict employees likely to quit, it might be possible to identify factors that contribute to their leaving. Because it is time-consuming and expensive to find, interview, and hire new employees, increasing employee retention will be beneficial to the company.

Familiarize yourself with the HR dataset

The dataset that you'll be using in this lab contains 15,000 rows and 10 columns for the variables listed below.

Note: you don't need to download any data to complete this lab. For more information about the data, refer to its source on [Kaggle](#).

Variable	Description
satisfaction_level	Employee-reported job satisfaction level [0-1]
last_evaluation	Score of employee's last performance review [0-1]
number_project	Number of projects employee contributes to
average_monthly_hours	Average number of hours employee worked per month
time_spend_company	How long the employee has been with the company (years)
Work_accident	Whether or not the employee experienced an accident while at work
left	Whether or not the employee left the company
promotion_last_5years	Whether or not the employee was promoted in the last 5 years
Department	The employee's department
salary	The employee's salary (U.S. dollars)



Reflect on these questions as you complete the plan stage.

- 1. Who are your stakeholders for this project?
 - 2. What are you trying to solve or accomplish?
 - 3. What are your initial observations when you explore the data?
 - 4. What resources do you find yourself using as you complete this stage?
(Make sure to include the links.)
 - 5. Do you have any ethical considerations in this stage?
-
- 1. The members of the HR department at Salifort Motors and the management team.
 - 2. I'm trying to discover what makes the employees dissatisfied enough to the point of leaving the company. In this process, I aim to generate insights that'll help me predict when an employee is at risk of leaving the company, so we can act before they take this decision.
 - 3. Most employees seem to have a reasonable satisfaction level, and a good evaluation level. Their average monthly hour can vary a lot, and might be a reason behind lower satisfaction levels. Employees with low salaries and those that didn't receive any promotions during the last 5 years may also be at risk of leaving.

- 4. The Sailfort Motors HR dataset, available on Kaggle (<https://www.kaggle.com/giripujar/hr-analytics>), the documentation of each package I'm using (pandas (<https://pandas.pydata.org/docs/>), seaborn(<https://seaborn.pydata.org/>), scikitlearn(<https://scikit-learn.org/stable/documentation.html>) and matplotlib(<https://matplotlib.org/stable/contents.html>)) and the ACM code of ethics(<https://www.acm.org/code-of-ethics>).
- 5. Some employees seem to be working too many hours, upwards of 260-300+ hours monthly on average. We also need to make sure there is as little bias as possible during the next stages.

Step 1. Install and import packages

- Install packages
- Import packages
- Load dataset

```
In [1]: !pip3 install numpy pandas seaborn matplotlib xgboost scikit-learn
```

Requirement already satisfied: numpy in c:\users\yanca\anaconda3\lib\site-packages (1.26.4)
Requirement already satisfied: pandas in c:\users\yanca\anaconda3\lib\site-packages (2.2.2)
Requirement already satisfied: seaborn in c:\users\yanca\anaconda3\lib\site-packages (0.13.2)
Requirement already satisfied: matplotlib in c:\users\yanca\anaconda3\lib\site-packages (3.8.4)
Requirement already satisfied: xgboost in c:\users\yanca\anaconda3\lib\site-packages (2.1.2)
Requirement already satisfied: scikit-learn in c:\users\yanca\anaconda3\lib\site-packages (1.4.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\yanca\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\yanca\anaconda3\lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\yanca\anaconda3\lib\site-packages (from pandas) (2023.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\yanca\anaconda3\lib\site-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\yanca\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\yanca\anaconda3\lib\site-packages (from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\yanca\anaconda3\lib\site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\yanca\anaconda3\lib\site-packages (from matplotlib) (23.2)
Requirement already satisfied: pillow>=8 in c:\users\yanca\anaconda3\lib\site-packages (from matplotlib) (10.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\yanca\anaconda3\lib\site-packages (from matplotlib) (3.0.9)
Requirement already satisfied: scipy in c:\users\yanca\anaconda3\lib\site-packages (from xgboost) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in c:\users\yanca\anaconda3\lib\site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\yanca\anaconda3\lib\site-packages (from scikit-learn) (2.2.0)
Requirement already satisfied: six>=1.5 in c:\users\yanca\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)

Import packages

```
In [2]: # Import packages

# Basic packages
import numpy as np
import pandas as pd

# Visualization packages
import seaborn as sns
import matplotlib.pyplot as plt

# Data modeling packages
from xgboost import XGBClassifier
```

```

from xgboost import XGBRegressor
from xgboost import plot_importance

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# For metrics and helpful functions
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, \
f1_score, confusion_matrix, ConfusionMatrixDisplay, classification_report
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.tree import plot_tree
from sklearn.metrics import roc_curve, auc

# For displaying all of the columns in dataframes
pd.set_option('display.max_columns', None)

```

Load dataset

Pandas is used to read a dataset called `HR_capstone_dataset.csv`. As shown in this cell, the dataset has been automatically loaded in for you. You do not need to download the .csv file, or provide more code, in order to access the dataset and proceed with this lab. Please continue with this activity by completing the following instructions.

```

In [3]: # Run this cell to import your data and load the dataset into a dataframe
df0 = pd.read_csv("c:/Users/yanca/Documents/HR_capstone_dataset.csv")

# Display first few rows of the dataframe
df0.head(10)

```

```

Out[3]:

```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours
0	0.38	0.53	2	157
1	0.80	0.86	5	262
2	0.11	0.88	7	272
3	0.72	0.87	5	223
4	0.37	0.52	2	159
5	0.41	0.50	2	153
6	0.10	0.77	6	247
7	0.92	0.85	5	259
8	0.89	1.00	5	224
9	0.42	0.53	2	142

Step 2. Data Exploration (Initial EDA and data cleaning)

- Understand your variables
- Clean your dataset (missing data, redundant data, outliers)

Gather basic information about the data

```
In [4]: # Gather basic information about the data
df0.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   satisfaction_level      14999 non-null  float64
1   last_evaluation        14999 non-null  float64
2   number_project         14999 non-null  int64
3   average_monthly_hours  14999 non-null  int64
4   time_spend_company     14999 non-null  int64
5   Work_accident          14999 non-null  int64
6   left                   14999 non-null  int64
7   promotion_last_5years  14999 non-null  int64
8   Department             14999 non-null  object
9   salary                 14999 non-null  object
dtypes: float64(2), int64(6), object(2)
memory usage: 1.1+ MB
```

Gather descriptive statistics about the data

```
In [5]: # Gather descriptive statistics about the data
df0.describe()
```

```
Out[5]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours
count	14999.000000	14999.000000	14999.000000	14999.000000
mean	0.612834	0.716102	3.803054	201.0503
std	0.248631	0.171169	1.232592	49.9430
min	0.090000	0.360000	2.000000	96.0000
25%	0.440000	0.560000	3.000000	156.0000
50%	0.640000	0.720000	4.000000	200.0000
75%	0.820000	0.870000	5.000000	245.0000
max	1.000000	1.000000	7.000000	310.0000

Rename columns

As a data cleaning step, rename the columns as needed. Standardize the column names so that they are all in `snake_case`, correct any column names that are misspelled, and make column names more concise as needed.

```
In [6]: # Display all column names
df0.columns
```

```
Out[6]: Index(['satisfaction_level', 'last_evaluation', 'number_project',
              'average_monthly_hours', 'time_spend_company', 'Work_accident', 'left',
              'promotion_last_5years', 'Department', 'salary'],
              dtype='object')
```

```
In [7]: # Rename columns as needed
df0 = df0.rename(columns={'Department': 'department',
                          'average_monthly_hours': 'average_monthly_hours',
                          'time_spend_company': 'tenure',
                          'Work_accident': 'work_accident'})

# Display all column names after the update
df0.columns
```

```
Out[7]: Index(['satisfaction_level', 'last_evaluation', 'number_project',
              'average_monthly_hours', 'tenure', 'work_accident', 'left',
              'promotion_last_5years', 'department', 'salary'],
              dtype='object')
```

Check missing values

Check for any missing values in the data.

```
In [8]: # Check for missing values
df0.isna().sum()
```

```
Out[8]: satisfaction_level    0
        last_evaluation      0
        number_project       0
        average_monthly_hours 0
        tenure               0
        work_accident        0
        left                 0
        promotion_last_5years 0
        department           0
        salary               0
        dtype: int64
```

Check duplicates

Check for any duplicate entries in the data.

```
In [9]: # Check for duplicates
df0.duplicated().sum()
```

Out[9]: 3008

```
In [10]: # Inspect some rows containing duplicates as needed
df0[df0.duplicated()].head(100)
```

Out[10]:

	satisfaction_level	last_evaluation	number_project	average_monthly_h
396	0.46	0.57	2	
866	0.41	0.46	2	
1317	0.37	0.51	2	
1368	0.41	0.52	2	
1461	0.42	0.53	2	
...	...	...	...	
12086	0.17	0.84	5	
12087	0.44	0.45	2	
12088	0.37	0.57	2	
12089	0.10	0.79	6	
12090	0.40	0.50	2	

100 rows × 10 columns

Seeing as we have 15000 rows and 10 columns, it is very unlikely that multiple employees have submitted the exact same information on their entries, especially considering how many of these contain continuous variables.

```
In [11]: # Drop duplicates and save resulting dataframe in a new variable as needed
df1 = df0.drop_duplicates(keep='first')

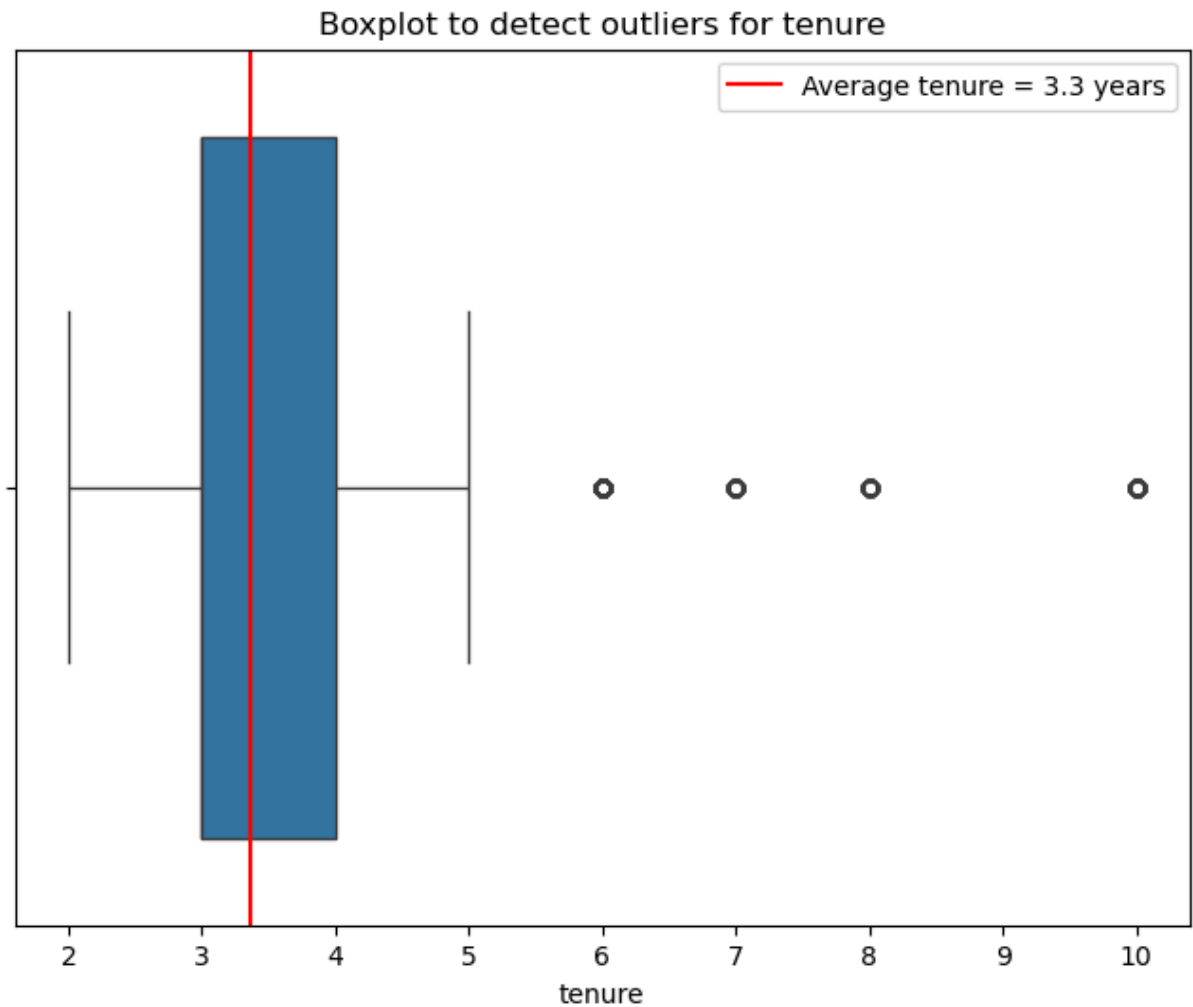
# Display first few rows of new dataframe as needed
df0.head(10)
```

Out[11]:	satisfaction_level	last_evaluation	number_project	average_monthly_hours
0	0.38	0.53	2	157
1	0.80	0.86	5	262
2	0.11	0.88	7	272
3	0.72	0.87	5	223
4	0.37	0.52	2	159
5	0.41	0.50	2	153
6	0.10	0.77	6	247
7	0.92	0.85	5	259
8	0.89	1.00	5	224
9	0.42	0.53	2	142

Check outliers

Check for outliers in the data.

```
In [12]: # Create a boxplot to visualize the distribution of `tenure` and detect any
plt.figure(figsize=(8,6))
plt.title('Boxplot to detect outliers for tenure', fontsize=12)
sns.boxplot(data = df1, x = 'tenure')
plt.axvline(x = df1['tenure'].mean(), color = 'red', label = 'Average tenure')
plt.legend();
```



The vast majority of employees are around the average of 3.3 years. There are however, some big outliers, with 6, 7, 8 and even 10 years of tenure.

```
In [13]: # Determine the number of rows containing outliers

# Calculates the 25th percentile of the 'tenure' column
percentile_25 = df1['tenure'].quantile(0.25)

# Calculates the 75th percentile of the 'tenure' column
percentile_75 = df1['tenure'].quantile(0.75)

# Calculate IQR
iqr = percentile_75 - percentile_25

upper_limit = percentile_75 + 1.5 * iqr
lower_limit = percentile_25 - 1.5 * iqr
print('Lower limit:', lower_limit, 'years')
print('Upper limit:', upper_limit, 'years')

# Identify subset of data containing outliers in `tenure` and display the count
outliers = df1[(df1['tenure'] > upper_limit) | (df1['tenure'] < lower_limit)]
print('Number of outliers:', len(outliers))
```

Lower limit: 1.5 years
Upper limit: 5.5 years
Number of outliers: 824

Certain types of models are more sensitive to outliers than others. When you get to the stage of building your model, consider whether to remove outliers, based on the type of model you decide to use.

pAce: Analyze Stage

- Perform EDA (analyze relationships between variables)



Reflect on these questions as you complete the analyze stage.

- 1. What did you observe about the relationships between variables?
 - 2. What do you observe about the distributions in the data?
 - 3. What transformations did you make with your data? Why did you chose to make those decisions?
 - 4. What are some purposes of EDA before constructing a predictive model?
 - 5. What resources do you find yourself using as you complete this stage? (Make sure to include the links.)
 - 6. Do you have any ethical considerations in this stage?
-
- 1. The satisfaction level column had the highest negative correlation with the left column. This means that employees with lower satisfaction levels are likely to leave. The vast majority of employees who left were not promoted, meaning this may be another big factor behind their decision. Most of the employees who left fall in 2 categories: Overworked and underworked. They are eihter working much more hours averagely per month than average (240-300 hours), or considerably less (120-150 hours). This also reflects on the amount of projects they work per month. Most employees that had 2 projects in a month left, and almost all of the employees that had 7 projects in a month left.
 - 2. There are few outliers in the data and there is no significant difference between the departments. The 'salary' variable is categorical, and listed as low, medium and high. This may need to be dummy encoded later if it's used for modeling. The 'promotion_last_5years', 'left' and 'Work_accident' variables are binary with imbalanced distributions. Seeing as we are intrested in discovering the reasons behind employees leaving the company, a logistic regression may help us achieve this goal.

- 3. Some columns were renamed in order to standardize them. Outliers were detected and can be easily removed if deemed necessary later and duplicates were removed, as they were most likely caused by human error.
- 4. EDA helps us deepen our understanding of the data and the relationship between variables. We also perform additional data cleaning and feature engineering during EDA, as we slowly figure out new aspects of the dataset and close in our decision for modeling construction. This process can be essential depending on the chosen model. We also need to account for bias and fairness, as the data needs to be representative and some models can tend towards bias.
- 5. The documentation of the pandas (<https://pandas.pydata.org/docs/>), seaborn(<https://seaborn.pydata.org/>) and matplotlib(<https://matplotlib.org/stable/contents.html>) packages, as well as the Stack Overflow website (<https://stackoverflow.com/>) for troubleshooting.
- 6. Some employees seem to be working too many hours, upwards of 260-300+ hours monthly on average. We also need to make sure there is as little bias as possible during the next stages, and the data is both safely secured and adequately used.

Step 2. Data Exploration (Continue EDA)

Begin by understanding how many employees left and what percentage of all employees this figure represents.

```
In [14]: # Get numbers of people who left vs. stayed
print(df1['left'].value_counts(sort = True))

# Get percentages of people who left vs. stayed
print(df1['left'].value_counts(normalize = True)*100)
```

```
left
0    10000
1     1991
Name: count, dtype: int64

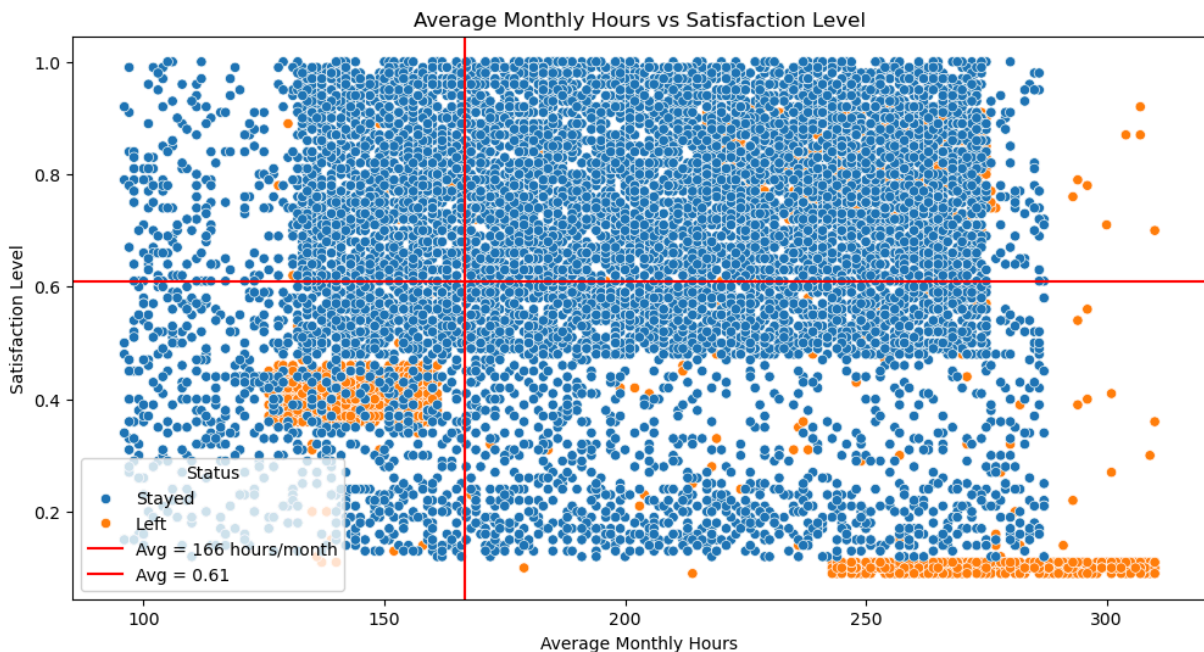
left
0    83.39588
1    16.60412
Name: proportion, dtype: float64
```

Data visualizations

Now, examine variables that you're interested in, and create plots to visualize relationships between variables in the data.

```
In [15]: # Create a plot of Average Monthly Hours vs Satisfaction Level
plt.figure(figsize=(12, 6))
sns.scatterplot(data=df1, x='average_monthly_hours', y='satisfaction_level',
plt.title('Average Monthly Hours vs Satisfaction Level')
plt.xlabel('Average Monthly Hours')
plt.ylabel('Satisfaction Level')
line_v = plt.axvline(x=166.67, color='red')
line_h = plt.axhline(y=0.61, color='red')

# Edit the legend
handles, labels = plt.gca().get_legend_handles_labels()
new_labels = ['Stayed', 'Left', 'Avg = 166 hours/month', 'Avg = 0.61']
handles.extend([line_v, line_h])
plt.legend(handles=handles, labels=new_labels, title='Status');
```

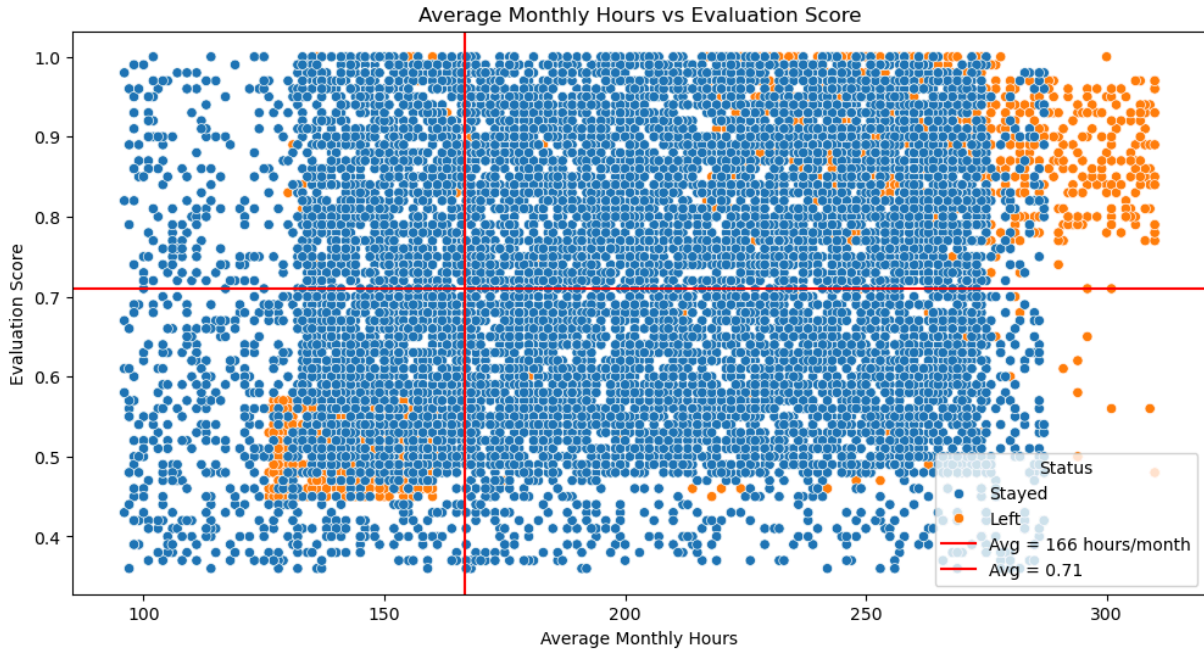


This scatterplot showcases a clear, very low satisfaction score between multiple employees that worked a lot of monthly hours (240+ hours). This could be a big reason behind their decision to leave. On the other hand, we have another group of employees which left that worked below average hours (around 120-160 hours), and also have a low satisfaction score. Most of the other employees that left seem scatter randomly around the plot. The majority of the employees that stay have a good satisfaction score.

```
In [16]: # Create a plot of Average Monthly Hours vs Evaluation Score
plt.figure(figsize=(12, 6))
sns.scatterplot(data=df1, x='average_monthly_hours', y='last_evaluation', hu
plt.title('Average Monthly Hours vs Evaluation Score')
plt.xlabel('Average Monthly Hours')
plt.ylabel('Evaluation Score')
line_v = plt.axvline(x=166.67, color='red')
line_h = plt.axhline(y=0.71, color='red')

# Edit the legend
```

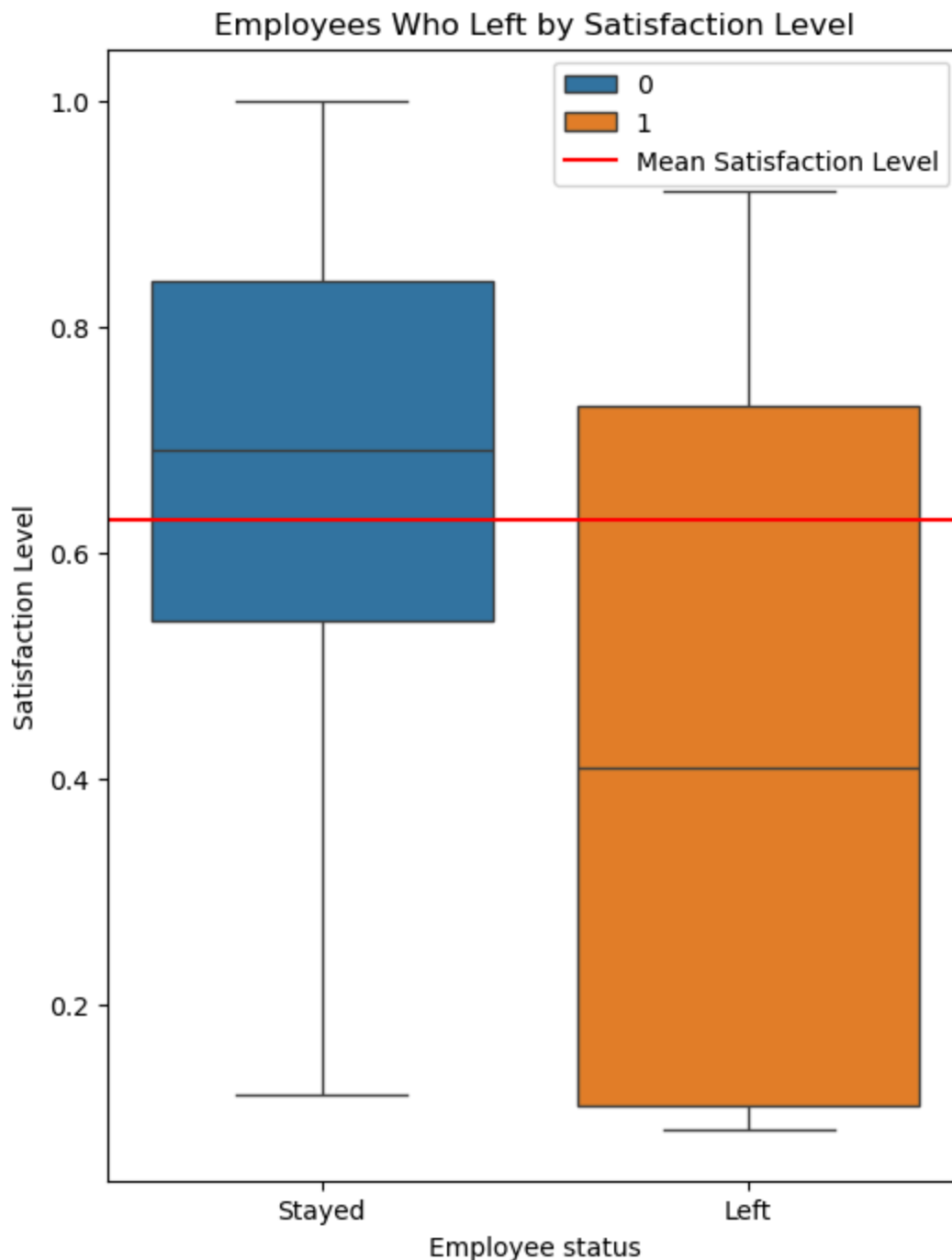
```
handles, labels = plt.gca().get_legend_handles_labels()
new_labels = ['Stayed', 'Left', 'Avg = 166 hours/month', 'Avg = 0.71']
handles.extend([line_v, line_h])
plt.legend(handles=handles, labels=new_labels, title='Status');
```



This plot has some similarities between the Average Monthly Hours vs Satisfaction Level plot. We can see that there is a group of employees on the bottom left corner that also left, and have below average monthly hours, that have a below average evaluation score. The employees that worked the longest average hours however, whereas in the previous plot most of them had extremely low satisfaction score, in this plot they have a very high evaluation score on average. It does seem however, that there is no correlation between average monthly hours and evaluation score for the employees that didn't leave.

```
In [18]: # Create a plot of Employees Who Left by Satisfaction Level

plt.figure(figsize=(6, 8))
sns.boxplot(data=df1, x='left', y='satisfaction_level', hue='left')
plt.title('Employees Who Left by Satisfaction Level')
plt.xlabel('Employee status')
plt.ylabel('Satisfaction Level')
plt.xticks(ticks=[0, 1], labels=['Stayed', 'Left'])
plt.axhline(y=df1['satisfaction_level'].mean(), color='red', label='Mean Sat')
plt.legend();
```



We can see a clear difference between the average satisfaction score of employees who left vs employees who stayed. Most of the employees who left have a satisfaction score lower than average, and they reach very low scores, while most of the employees that stayed have higher than average satisfaction scores, with a low variance.

```
In [ ]: fig, ax = plt.subplots(1, 2, figsize = (18,8))

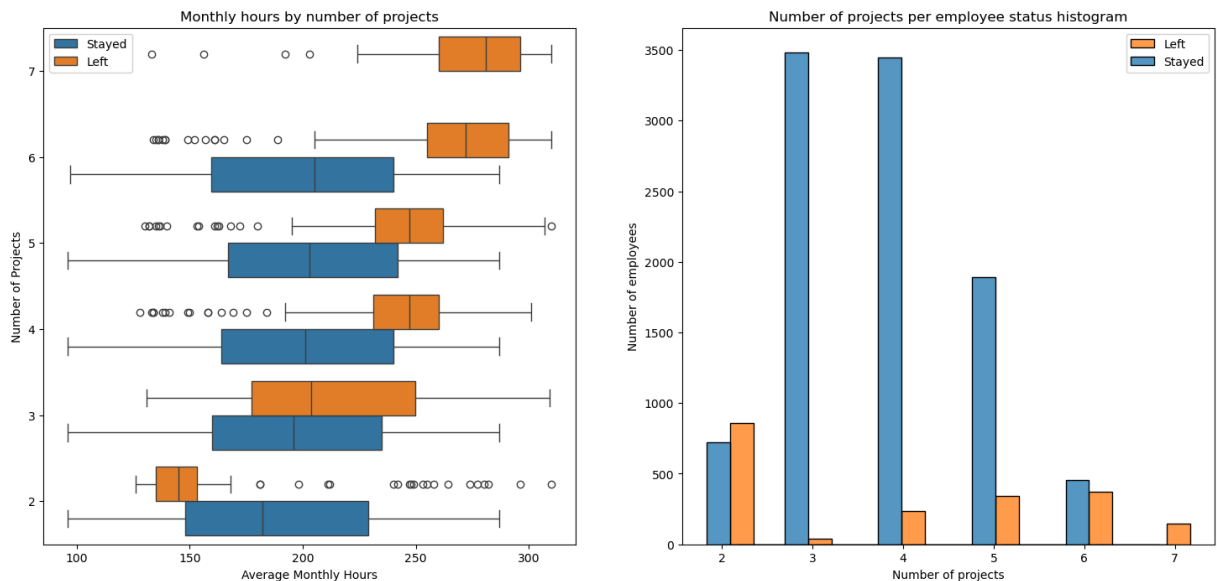
# Create boxplot showing monthly hours by number of projects comparing emplc
sns.boxplot(data=df1, x='average_monthly_hours', y='number_project', hue='le
ax[0].invert_yaxis()
ax[0].set_title('Monthly hours by number of projects')
```

```

ax[0].set_xlabel('Average Monthly Hours')
ax[0].set_ylabel('Number of Projects')
# Customize the legend for the boxplot
handles, labels = ax[0].get_legend_handles_labels()
ax[0].legend(handles=handles, labels=['Stayed', 'Left'])

# Create histogram showing distribution of `number_project`, comparing empl
tenure_stay = df1[df1['left']==0]['number_project']
tenure_left = df1[df1['left']==1]['number_project']
sns.histplot(data=df1, x='number_project', hue='left', multiple='dodge', shr
plt.xlabel('Number of projects')
plt.ylabel('Number of employees')
plt.legend(labels=['Left', 'Stayed'])
ax[1].set_title('Number of projects per employee status histogram');

```



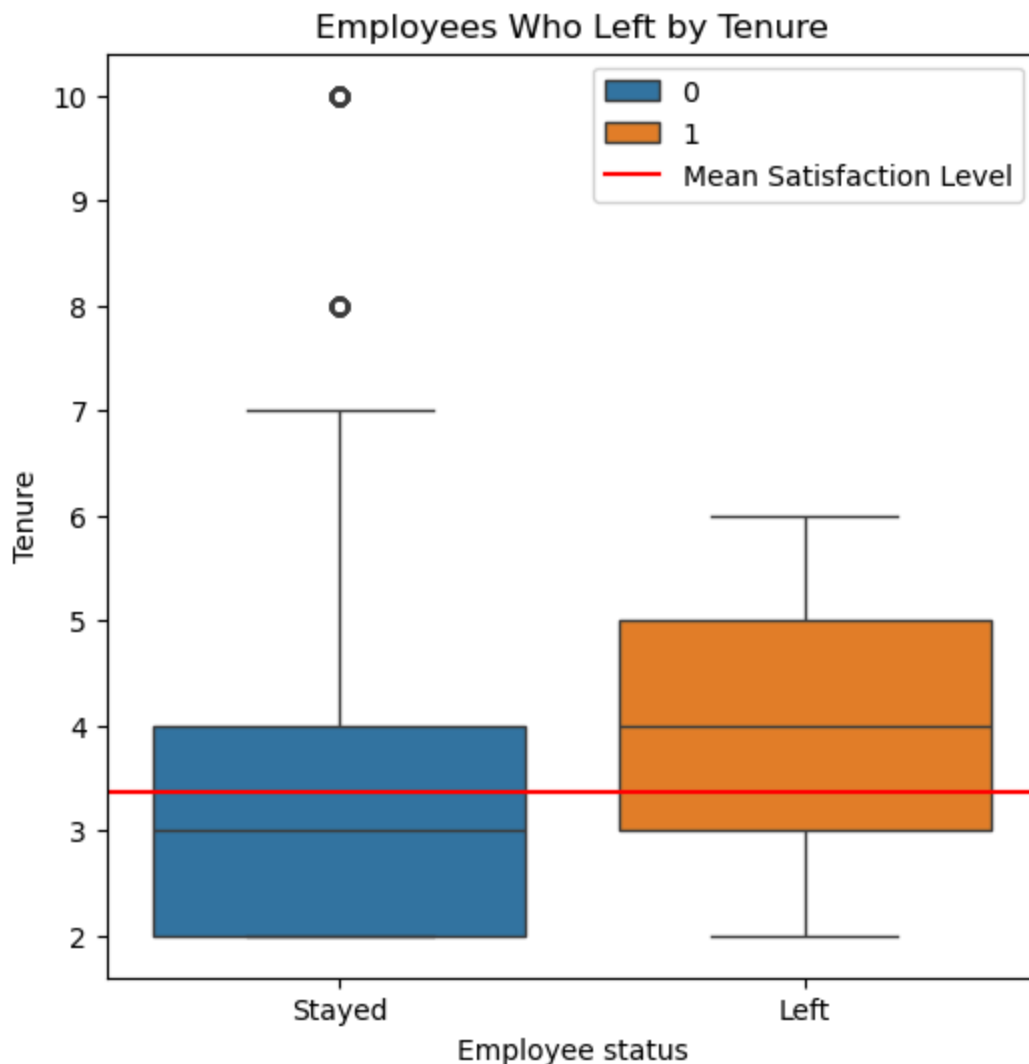
The boxplot and the histogram can showcase a relationship between the amount of average monthly hours and the number of monthly projects employees work. The higher their average amount of average monthly hours, the higher their amount of monthly projects, and the lower their average amount of average monthly hours, the lower their amount of monthly projects. We can also see that the vast majority of employees who worked 7 projects in a month left, and most of the employees that worked 2 projects in a month also left. This may indicate that the main cause of these 2 groups of employees leaving is that they are either overworked, or working less than what they desire.

```

In [ ]: # Create a plot of Employees Who Left by Tenure

plt.figure(figsize=(6, 6))
sns.boxplot(data=df1, x='left', y='tenure', hue = 'left')
plt.title('Employees Who Left by Tenure')
plt.xlabel('Employee status')
plt.ylabel('Tenure')
plt.xticks(ticks=[0, 1], labels=['Stayed', 'Left'])
plt.axhline(y=df1['tenure'].mean(), color='red', label='Mean Satisfaction Le
plt.legend();

```



Similar to the previous plot, we can see a clear difference in the tenure between employees who left and those that stayed. Most of the employees who left worked between 3,5 - 5 years, while the majority that didn't leave worked less than 3,5 years. It is also worth noting that there are some outliers who worked between 5-10 years who stayed in the company.

```
In [27]: fig, ax = plt.subplots(1, 2, figsize=(18, 8))

# Define custom legend handles
mean_line = plt.Line2D([0], [0], color='red', label='mean line')

# Plot 1: Employees Who Left by Evaluation Score
sns.boxplot(data=df1, x='left', y='last_evaluation', hue='left', ax=ax[0])
ax[0].set_title('Employees Who Left by Evaluation Score')
ax[0].set_xlabel('Employee status')
ax[0].set_ylabel('Evaluation Score')
ax[0].axhline(y=df1['last_evaluation'].mean(), color='red')

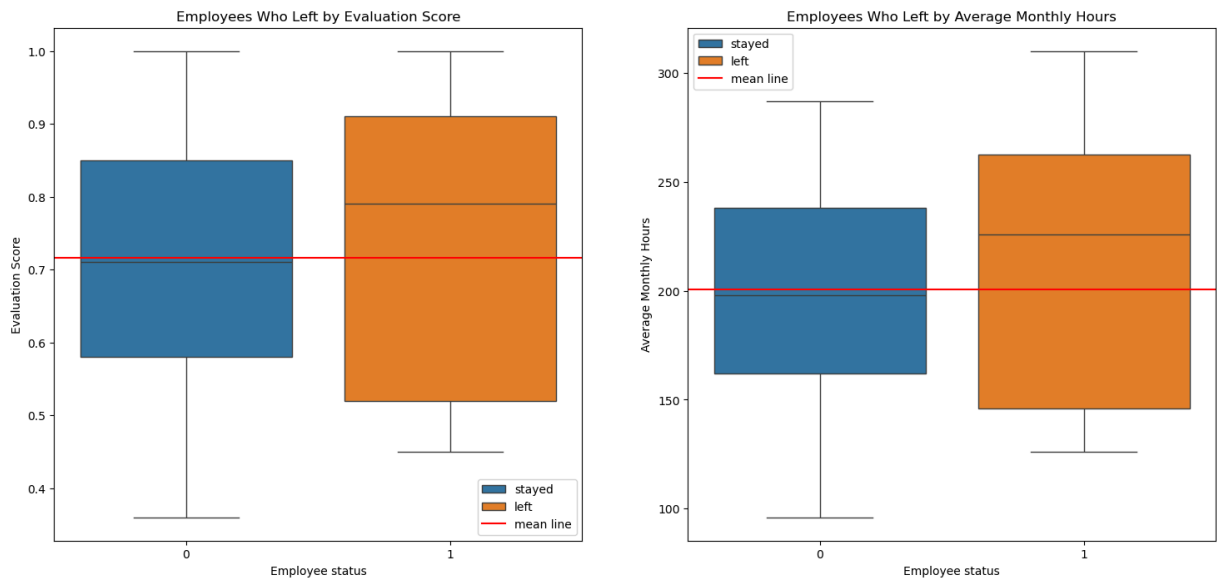
# Plot 2: Employees Who Left by Average Monthly Hours
sns.boxplot(data=df1, x='left', y='average_monthly_hours', hue='left', ax=ax[1])
ax[1].set_title('Employees Who Left by Average Monthly Hours')
```

```

ax[1].set_xlabel('Employee status')
ax[1].set_ylabel('Average Monthly Hours')
ax[1].axhline(y=df1['average_monthly_hours'].mean(), color='red')

# Adjust legend in both plots
for axis in ax:
    handles, _ = axis.get_legend_handles_labels()
    handles[0].set_label('stayed')
    handles[1].set_label('left')
    handles.append(mean_line)
    axis.legend(handles=handles)

```



In the first plot, there is higher variance between the evaluation score of employees who left compared to those who stayed. Other than that, they seem pretty similar. It might be the case that employees who left and worked 240+ hours monthly on average were the one with higher than average evaluation score, while the ones that left and worked 110-150 hours monthly on average were the ones with lower than average evaluation scores. The second plot is very similar to the previous one, showing more evidence that the evaluation scores from employees who left might have a strong correlation with their average monthly hours.

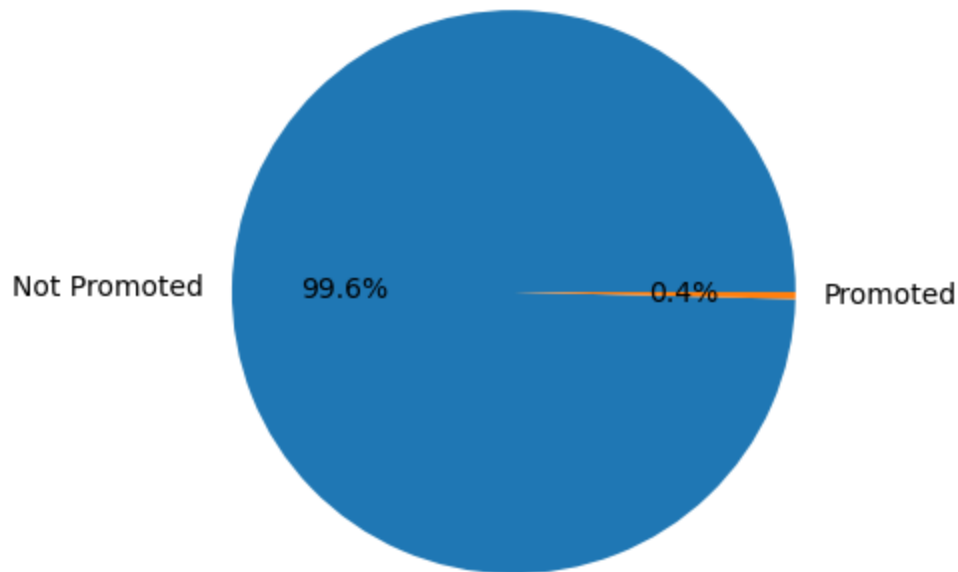
```

In [21]: # Create a plot of promoted vs not promoted employees who left

df_left = df1[df1['left'] == 1]
promotion_last_5years_counts = df_left['promotion_last_5years'].value_counts
labels = ['Not Promoted', 'Promoted']
plt.figure(figsize = (4,4))
plt.pie(promotion_last_5years_counts, labels = labels, autopct='%1.1f%%')
plt.title('Promoted vs Not Promoted Employees Who Left')
plt.axis('equal');

```

Promoted vs Not Promoted Employees Who Left

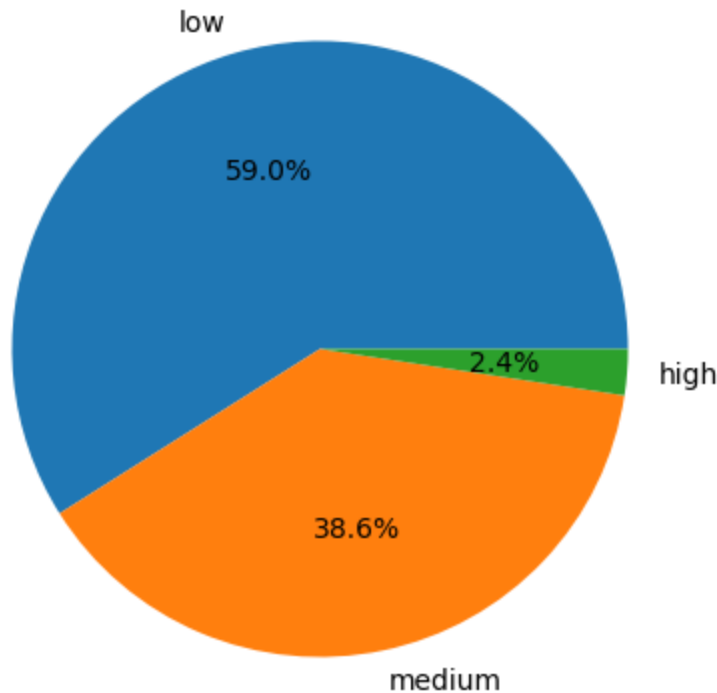


Seeing as the vast majority of employees who left were not promoted, despite the fact that their average tenure is longer than those who stayed, this may be another key reason why they left. Most of these employees may have been expecting promotions after 3,5 years of work, and after not getting one, they sought out different positions in other companies.

```
In [22]: # Create a plot of employees who left by salary

salary_counts = df_left['salary'].value_counts()
plt.figure(figsize = (5,5))
plt.pie(salary_counts, labels = salary_counts.index, autopct='%1.1f%%')
plt.title('Employees who left by salary');
```

Employees who left by salary



Most of the employees who left had either low or medium salaries. The employees with low salary may have left because of that. The employees with medium/high salary on the other hand, may have had different reasons to leave.

```
In [23]: fig, ax = plt.subplots(1, 2, figsize=(18, 8))

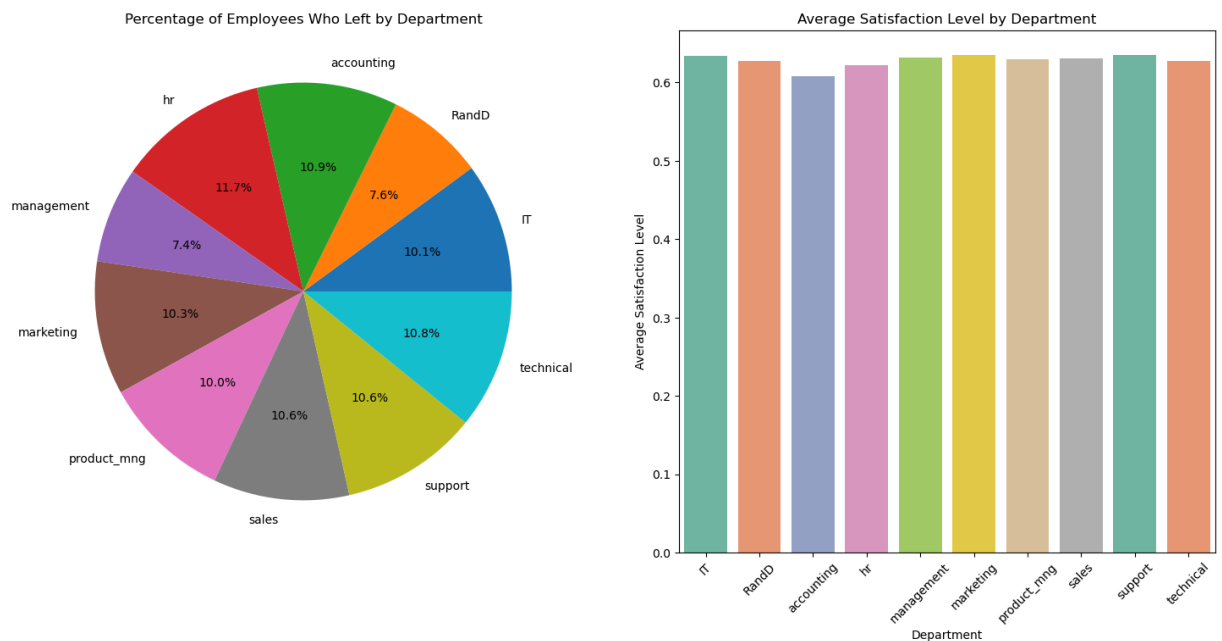
# Create a pie chart of percentage employees who left by department
department_total = df1['department'].value_counts()
department_left_proportional = df_left['department'].value_counts() / department_total
ax[0].pie(department_left_proportional, labels=department_left_proportional.index, autopct='%1.1f%%')
ax[0].set_title('Percentage of Employees Who Left by Department')

# Create a bar plot of average satisfaction level by department
avg_satisfaction = df1.groupby('department')['satisfaction_level'].mean()
sns.barplot(x=avg_satisfaction.index, y=avg_satisfaction.values, palette='Set2')
ax[1].set_title('Average Satisfaction Level by Department')
ax[1].set_xlabel('Department')
ax[1].set_ylabel('Average Satisfaction Level')
ax[1].tick_params(axis='x', rotation=45);
```

C:\Users\yanca\AppData\Local\Temp\ipykernel_17164\2020139924.py:11: FutureWarning:

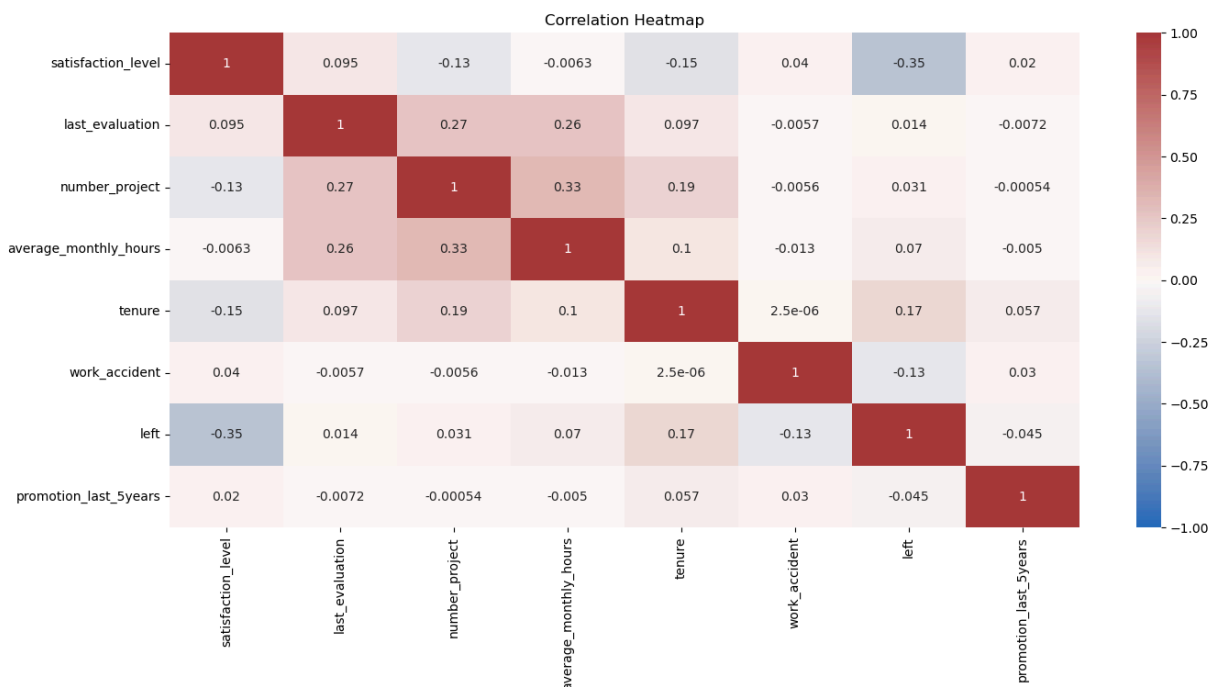
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=avg_satisfaction.index, y=avg_satisfaction.values, palette='Set2', ax=ax[1])
```



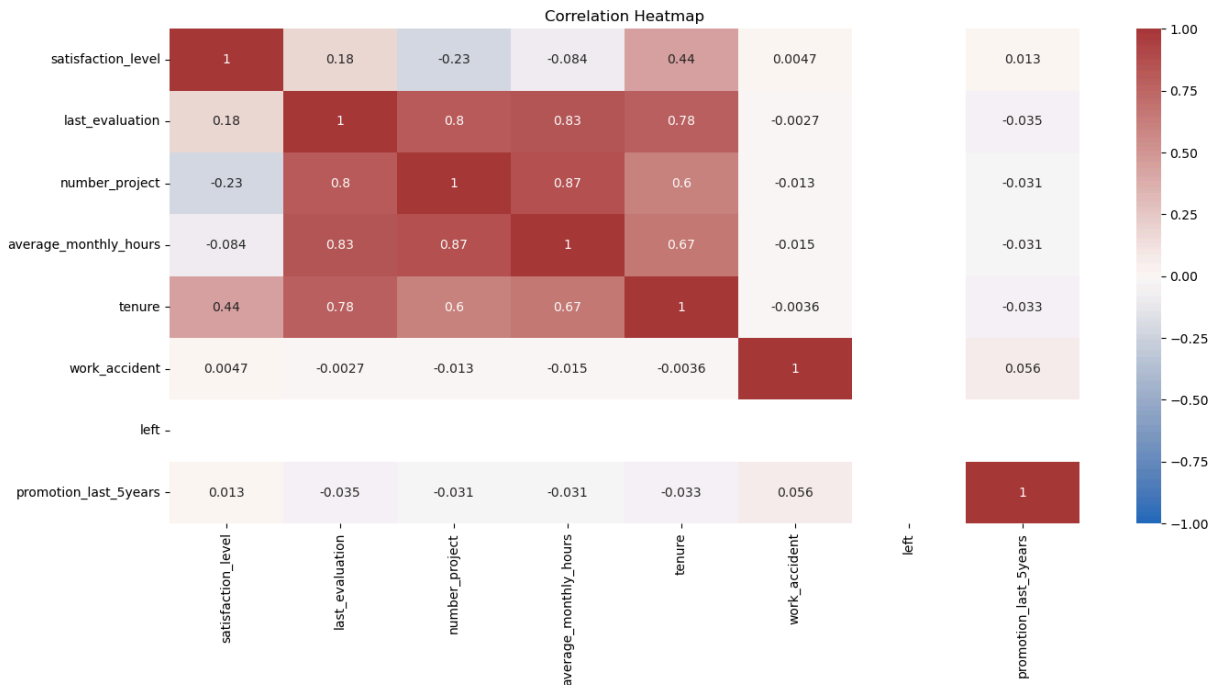
Analyzing the pie chart, there doesn't seem to exist any clear distinction between the rate of employees leaving per department. Most of them have a similar rate, at around 10%. Following the trend for the previous chart, there isn't a significant difference between the satisfaction level by department showcased by the bar plot. Most likely, the main reason behind employees leaving lay outside their specifics departments.

```
In [24]: # Create a plot of Correlation Heatmap
plt.figure(figsize=(16, 7))
heatmap = sns.heatmap(df1.select_dtypes(include='number').corr(), vmin=-1, v
heatmap.set_title('Correlation Heatmap');
```



As we are trying to discover the reasons behind employees leaving the company, the variables in this heatmap that have the highest correlations (the pink ones and the dark blue one) are gonna be used for modeling later.

```
In [25]: # Create a plot of Correlation Heatmap
plt.figure(figsize=(16, 7))
heatmap = sns.heatmap(df_left.select_dtypes(include='number').corr(), vmin=-
heatmap.set_title('Correlation Heatmap');
```



Seeing as this heatmap only contains data on employees that left, we can see that there is a very high multicollinearity between some variables (`tenure` , `average_monthly_hours` , `number_project` and `last_evaluation`), indicating that multiple factors are behind the employees decision to leave, but they are also very closely related to each other.

Insights

- The `satisfaction_level` column had the highest negative correlation with the `left` column. This means that employees with lower satisfaction levels are likely to leave. Most of the employees who left had below average satisfaction levels.
- The vast majority of employees who left were not promoted, meaning this may be another big factor behind their decision.
- Most of the employees who left fall in 2 categories: Overworked and underworked. They are either working much more hours averagely per month than average (240-300 hours), or considerably less (120-150 hours). This also reflects on the amount of projects they work per month. Most

employees that had 2 projects in a month left, and almost all of the employees that had 7 projects in a month left.

- There is no significant difference between departments.
- When analyzing only the employees who left, some variables have very high multicollinearity, demonstrating that some of the employees that leave can be easily identified following their work behavior and history.

paCe: Construct Stage

- Determine which models are most appropriate
- Construct the model
- Confirm model assumptions
- Evaluate model results to determine how well your model fits the data



Recall model assumptions

Logistic Regression model assumptions

- Outcome variable is categorical: The outcome variable is the `left` column, which is binary. **This assumption is met**
- Observations are independent of each other: No observation influences or predict any other observation. **This assumption is met**
- No severe multicollinearity among X variables: There is multicollinearity between some of the x variables. However, they are weak, meaning it should not affect the outcome too much. **This assumption is met**
- No extreme outliers: The extreme outliers will be removed before the model construction. **This assumption is met**
- Linear relationship between each X variable and the logit of the outcome variable. We can verify this assumption by seeing how each X variable affects the outcome variable. **This assumption is met**
- Sufficiently large sample size. The dataset contains 10 columns and almost 12.000 rows. **This assumption is met**



Reflect on these questions as you complete the constructing stage.

- 1. Do you notice anything odd?
- 2. Which independent variables did you choose for the model and why?
- 3. Are each of the assumptions met?

- 4. How well does your model fit the data?
 - 5. Can you improve it? Is there anything you would change about the model?
 - 6. What resources do you find yourself using as you complete this stage? (Make sure to include the links.)
 - 7. Do you have any ethical considerations in this stage?
-
- 1. There is an imbalance in the 'left' variable. It could affect the model's performance and lead to biased predictions. Because of this, the model's high accuracy might be misleading.
 - 2. Every variable that isn't the 'left' variable was chosen as an independent variable. This was done considering future applications of the model, as the relationship between the variables may change over time.
 - 3. The assumptions were not test for every variable, but they were mostly met.
 - 4. The accuracy of both models was very high (above 80%). The model fits the data very well.
 - 5. A Random Forest model may wield better results. However, due to how good the results of the Decision Tree model were and how it is a simpler and easier model to work with, the Decision Tree ended up being the final model.
 - 6. The documentation of the pandas (<https://pandas.pydata.org/docs/>) and the Scikit-learn package (<https://scikit-learn.org>) packages, as well as the Stack Overflow website (<https://stackoverflow.com/>) for troubleshooting.
 - 7. I have concerns over how the models developed will adress bias and overfitting when new data is introduced. There may be other models better suited for the task when it comes to ethical considerations.

Step 3. Model Building, Step 4. Results and Evaluation

- Fit a model that predicts the outcome variable using two or more independent variables
- Check model assumptions
- Evaluate the model

Identify the type of prediction task.

The goal for this analysis is to construct a model that aims to predict when employees will leave the company. This is a binary variable, where 0 indicates the employee is in the company, and 1 indicates the employee left.

Identify the types of models most appropriate for this task.

Considering the binary nature of the "left" variable, a logistic regression is a good model for predicting whether or not employees will leave. If the logistic regression results are deemed inaccurate or insufficient, then a decision tree or a random forest model will be used instead.

1st model: Logisitc Regression

A Logisitc Regression will be performed with the 'left' column as our observed variable, and every other column as features. A binomial logistic regression suits the task because it involves binary classification.

Before splitting the data into training and test data, we'll need to encode the 'salary' and 'departmente' variables, as those are non-numeric.

```
In [26]: # Create a copy of the dataframe
df_enc = df1.copy()

# Encode the salary column as a numerical category
df_enc['salary'] = df_enc['salary'].map({'low':0, 'medium':1, 'high':2})

# Dummy encode the `department` column
df_enc = pd.get_dummies(df_enc, drop_first=False)

# Display first 10 rows of the dataset
df_enc.head(10)
```

```
Out[26]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours
0	0.38	0.53	2	157
1	0.80	0.86	5	262
2	0.11	0.88	7	272
3	0.72	0.87	5	223
4	0.37	0.52	2	159
5	0.41	0.50	2	153
6	0.10	0.77	6	247
7	0.92	0.85	5	259
8	0.89	1.00	5	224
9	0.42	0.53	2	142

Considering how sensitive to outliers Logisitc Regressions can be, those will be removed for this model.

```
In [27]: # Create a new dataframe and remove outliers from 'tenure' that were detected
df_logreg = df_enc[(df_enc['tenure'] >= lower_limit) & (df_enc['tenure'] <=
```

Proceed to isolate the outcome variable and the features that'll be used.

```
In [28]: # Isolate the outcome variable
y = df_logreg['left']

# Isolate the features of interest
x = df_logreg.drop('left', axis=1)
```

Split the data into training data and test data in order to train and test the model. Seeing as the classes are unbalanced, we'll stratify based on the values in y.

```
In [29]: # Split the data into training data and test data
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.25,
```

Proceed with the model construction and fitting.

```
In [30]: # Construct a logistic regression model and fit it to the training dataset
log_clf = LogisticRegression(random_state=1, max_iter=1000).fit(x_train, y_train)

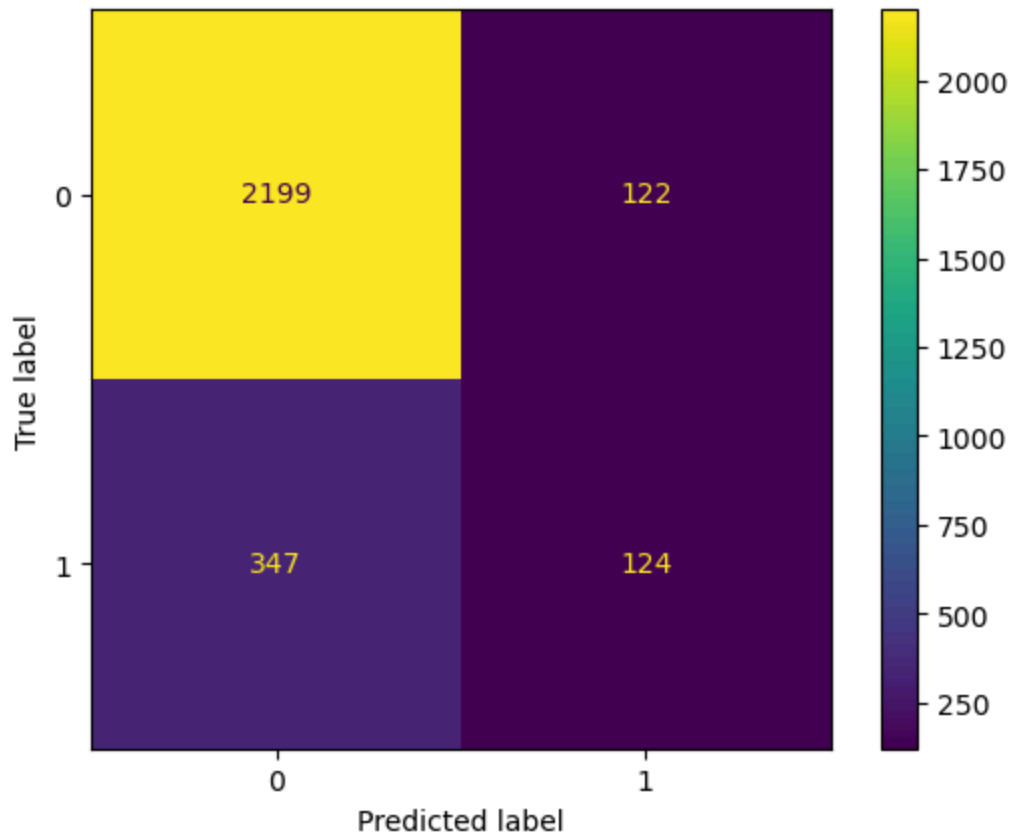
# Use the logistic regression model to get predictions on the test set
y_pred = log_clf.predict(x_test)
```

Create a confusion matrix to visualize the results of the model.

```
In [31]: # Compute values for confusion matrix
log_cm = confusion_matrix(y_test, y_pred, labels=log_clf.classes_)

# Create display of confusion matrix
log_disp = ConfusionMatrixDisplay(confusion_matrix=log_cm, display_labels=log_clf.classes_)

# Plot confusion matrix
log_disp.plot(values_format='');
```



- The top-left quadrant displays the number of true negatives (The number of people who stayed that the model accurately predicted would stay)
- The top-right quadrant displays the number of false positives (The number of people who stayed the model inaccurately predicted as leaving).
- The bottom-left quadrant displays the number of false negatives (The number of people who left that the model inaccurately predicted as leaving).
- The bottom-right quadrant displays the number of true positives (The number of people who left that the model accurately predicted as leaving)

A perfect model would yield all true negatives and true positives, and no false negatives or false positives.

```
In [32]: df_logreg['left'].value_counts(normalize=True)*100
```

```
Out[32]: left
0      83.146772
1      16.853228
Name: proportion, dtype: float64
```

The split on the data is imbalanced, but not severely so. Resampling the data could improve the balance, but seeing as it is so close to the balance of the original dataset, it shouldn't improve much.

Create a classification report that includes precision, recall, f1-score, and accuracy metrics to evaluate the performance of the logistic regression model.

```
In [33]: # Create classification report for logistic regression model
target_names = ['Predicted would not leave', 'Predicted would leave']
print(classification_report(y_test, y_pred, target_names=target_names))
```

	precision	recall	f1-score	support
Predicted would not leave	0.86	0.95	0.90	2321
Predicted would leave	0.50	0.26	0.35	471
accuracy			0.83	2792
macro avg	0.68	0.61	0.62	2792
weighted avg	0.80	0.83	0.81	2792

The classification report shows that the logistic regression model achieved a precision of 79%, recall of 82%, f1-score of 80% (all weighted averages), and accuracy of 83%. However, seeing as we are trying to predict when employees leave, then the scores are significantly lower. This means the logistic model is insufficient for our analysis. Considering this, we'll construct a Decision Tree model in hopes of achieving a better result.

2nd model: Decision Tree - round 1

The Decision Tree will likely be capable of achieving a better score than the Logistic Regression model, due to its flexibility in capturing complex relationships and handling various types of data.

```
In [37]: # Instantiate model
tree = DecisionTreeClassifier(random_state=1)

# Assign a dictionary of hyperparameters to search over
cv_params = {'max_depth': [4, 6, 8, None],
             'min_samples_leaf': [2, 5, 1],
             'min_samples_split': [2, 4, 6]
            }

# Assign a dictionary of scoring metrics to capture
scoring = {
    'accuracy': 'accuracy',
    'precision': 'precision',
    'recall': 'recall',
    'f1': 'f1',
    'roc_auc': 'roc_auc'
}

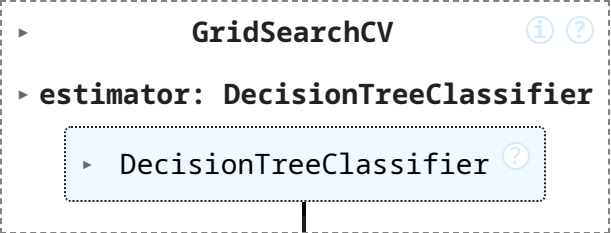
# Instantiate GridSearch
tree1 = GridSearchCV(tree, cv_params, scoring=scoring, cv=4, refit='roc_auc')
```

Get the time required for the model

```
In [38]: %%time
         tree1.fit(x_train, y_train)
```

CPU times: total: 5.81 s
Wall time: 6.1 s

```
Out[38]:
```



```
In [39]: # Check best parameters
         tree1.best_params_
```

```
Out[39]: {'max_depth': 4, 'min_samples_leaf': 5, 'min_samples_split': 2}
```

```
In [ ]: # Check best AUC score on CV
         tree1.best_score_
```

This is a very strong AUC score, meaning the model can predict employees who will leave very well. Next, we'll write a function that will help you extract all the scores from the grid search.

```
In [40]: def make_results(model_name:str, model_object, metric:str):
        """
        Arguments:
            model_name (string): what you want the model to be called in the output
            model_object: a fit GridSearchCV object
            metric (string): precision, recall, f1, accuracy, or auc

        Returns a pandas df with the F1, recall, precision, accuracy, and auc score
        for the model with the best mean 'metric' score across all validation folds
        """

        # Create dictionary that maps input metric to actual metric name in GridSearchCV
        metric_dict = {'auc': 'mean_test_roc_auc',
                       'precision': 'mean_test_precision',
                       'recall': 'mean_test_recall',
                       'f1': 'mean_test_f1',
                       'accuracy': 'mean_test_accuracy'
                      }

        # Get all the results from the CV and put them in a df
        cv_results = pd.DataFrame(model_object.cv_results_)

        # Isolate the row of the df with the max(metric) score
        best_estimator_results = cv_results.iloc[cv_results[metric_dict[metric]].idxmax()]

        # Extract Accuracy, precision, recall, and f1 score from that row
```

```

auc = best_estimator_results.mean_test_roc_auc
f1 = best_estimator_results.mean_test_f1
recall = best_estimator_results.mean_test_recall
precision = best_estimator_results.mean_test_precision
accuracy = best_estimator_results.mean_test_accuracy

# Create table of results
table = pd.DataFrame()
table = pd.DataFrame({'model': [model_name],
                      'precision': [precision],
                      'recall': [recall],
                      'F1': [f1],
                      'accuracy': [accuracy],
                      'auc': [auc]
                      })

return table

```

```

In [41]: # Get all CV scores
tree1_cv_results = make_results('decision tree cv', tree1, 'auc')
tree1_cv_results

```

```

Out[41]:

```

	model	precision	recall	F1	accuracy	auc
0	decision tree cv	0.958962	0.908564	0.932991	0.97803	0.973133

All of these scores from the decision tree model are strong indicators of good model performance and better than the previous Logistic Regression. Recall that decision trees can be vulnerable to overfitting, and random forests avoid overfitting by incorporating multiple trees to make predictions. However, seeing as random forests require more computational power, take longer to complete and has a longer code, the decision tree will be the definitive model.

Decision Tree - round 2

There is a chance that there is some data leakage occurring. Data leakage is when you use data to train your model that should not be used during training, either because it appears in the test data or because it's not data that you'd expect to have when the model is actually deployed. Training a model with leaked data can give an unrealistic score that is not replicated in production. In this case, it's likely that the company won't have satisfaction levels reported for all of its employees. It's also possible that the average_monthly_hours column is a source of some data leakage. If employees have already decided upon quitting, or have already been identified by management as people to be fired, they may be working fewer hours.

The first round of decision tree and random forest models included all variables as features. This round will incorporate feature engineering to build improved models. We'll proceed by dropping satisfaction_level and creating a new feature

that roughly captures whether an employee is overworked. It will be a binary variable.

```
In [42]: # Drop `satisfaction_level` and save resulting dataframe in new variable
df2 = df_enc.drop('satisfaction_level', axis=1)

# Create `overworked` column. For now, it's identical to average monthly hours
df2['overworked'] = df2['average_monthly_hours']

# Inspect max and min average monthly hours values
print('Max hours:', df2['overworked'].max())
print('Min hours:', df2['overworked'].min())
```

```
Max hours: 310
Min hours: 96
```

166.67 is approximately the average number of monthly hours for someone who works 50 weeks per year, 5 days per week, 8 hours per day. We'll define overworked employees as those with over 175 average hours per month.

```
In [43]: # Define `overworked` as working > 175 hrs/week
df2['overworked'] = (df2['overworked'] > 175).astype(int)

# Drop the `average_monthly_hours` column
df2 = df2.drop('average_monthly_hours', axis=1)

# Display first 10 rows of resulting dataframe
df2.head(10)
```

```
Out[43]:
```

	last_evaluation	number_project	tenure	work_accident	left	promotion_last_year
0	0.53	2	3	0	1	
1	0.86	5	6	0	1	
2	0.88	7	4	0	1	
3	0.87	5	5	0	1	
4	0.52	2	3	0	1	
5	0.50	2	3	0	1	
6	0.77	6	4	0	1	
7	0.85	5	5	0	1	
8	1.00	5	5	0	1	
9	0.53	2	3	0	1	

Once again, isolate the outcome variable , the features that'll be used and split the training data and test data.

```
In [44]: # Isolate the outcome variable
y = df2['left']
```

```

# Select the features
x = df2.drop('left', axis=1)

# Create test data
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, st

```

Proceed with the second decision tree model.

```

In [47]: # Instantiate model
tree = DecisionTreeClassifier(random_state=0)

# Assign a dictionary of hyperparameters to search over
cv_params = {'max_depth': [4, 6, 8, None],
             'min_samples_leaf': [2, 5, 1],
             'min_samples_split': [2, 4, 6]
            }

# Assign a dictionary of scoring metrics to capture
scoring = {
    'accuracy': 'accuracy',
    'precision': 'precision',
    'recall': 'recall',
    'f1': 'f1',
    'roc_auc': 'roc_auc'
}

# Instantiate GridSearch
tree2 = GridSearchCV(tree, cv_params, scoring=scoring, cv=4, refit='roc_auc'

```

Get the time required for the model

```

In [48]: %%time
tree2.fit(x_train, y_train)

```

CPU times: total: 5.39 s
Wall time: 5.56 s

```

Out[48]:
GridSearchCV
└─ estimator: DecisionTreeClassifier
   └─ DecisionTreeClassifier

```

```

In [49]: # Check best params
tree2.best_params_

```

```

Out[49]: {'max_depth': 6, 'min_samples_leaf': 2, 'min_samples_split': 6}

```

```

In [50]: # Check best AUC score on CV
print("Round 2 - best score:", tree2.best_score_)
print("")

# Get all CV scores

```

```
tree2_cv_results = make_results('decision tree2 cv', tree2, 'auc')
print(tree1_cv_results)
print(tree2_cv_results)
```

Round 2 - best score: 0.9586752505340426

	model	precision	recall	F1	accuracy	auc
0	decision tree cv	0.958962	0.908564	0.932991	0.97803	0.973133
	model	precision	recall	F1	accuracy	auc
0	decision tree2 cv	0.856693	0.903553	0.878882	0.958523	0.958675

In [51]: tree1_cv_results

Out[51]:

	model	precision	recall	F1	accuracy	auc
0	decision tree cv	0.958962	0.908564	0.932991	0.97803	0.973133

In [52]: tree2_cv_results

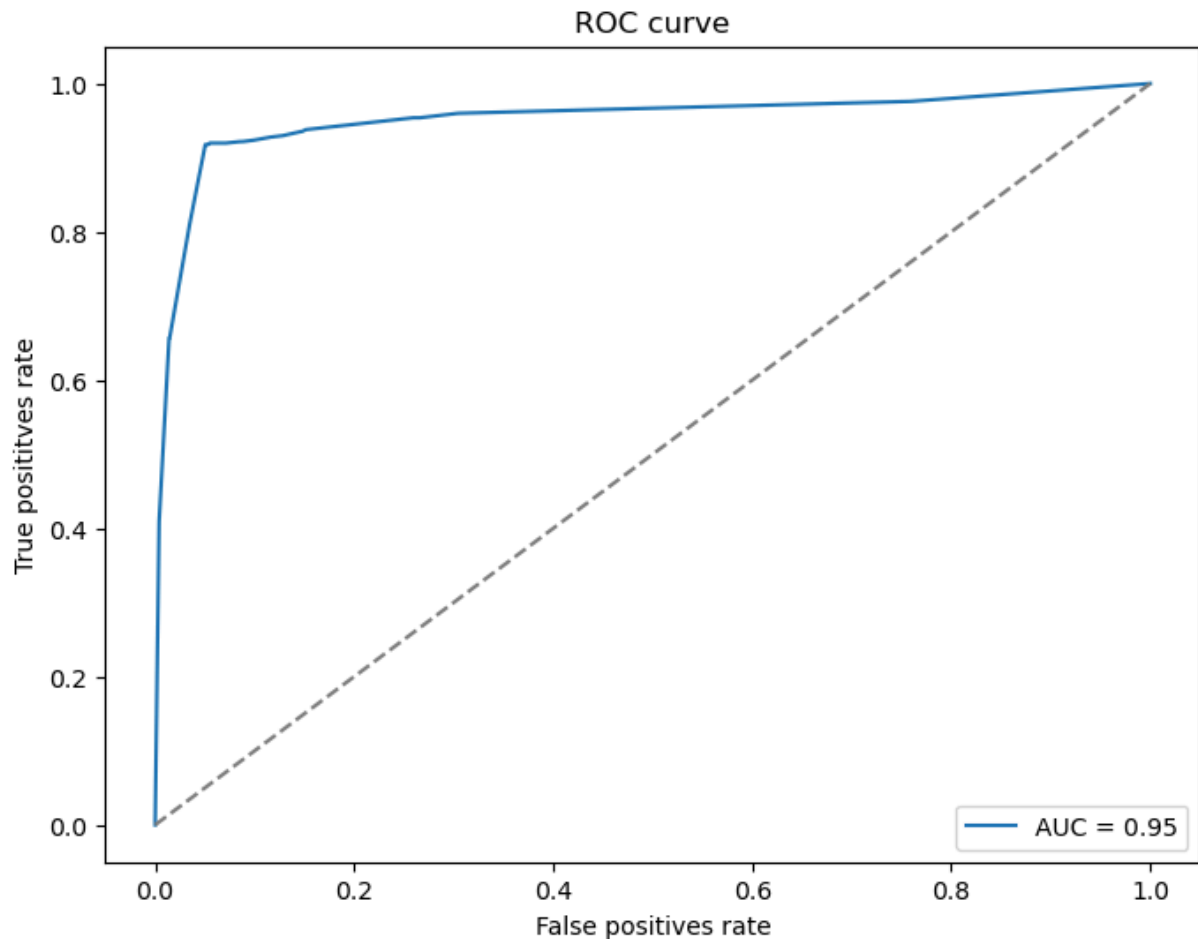
Out[52]:

	model	precision	recall	F1	accuracy	auc
0	decision tree2 cv	0.856693	0.903553	0.878882	0.958523	0.958675

Despite some of the scores being lower than the previous round (which is expected considering the fact that there are fewer features), the overall scores are still very good.

```
In [55]: y_score = tree2.predict_proba(x_test)[: , 1]
fpr, tpr, thresholds = roc_curve(y_test, y_score)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False positives rate')
plt.ylabel('True positivtes rate')
plt.title('ROC curve')
plt.legend(loc='lower right')
plt.show()
```



An ROC AUC score of 0.95 indicates that the model has a 95% probability of ranking a random positive example higher than a random negative example.

pacE: Execute Stage

- Interpret model performance and results
- Share actionable steps with stakeholders



Recall evaluation metrics

- **AUC** is the area under the ROC curve; it's also considered the probability that the model ranks a random positive example more highly than a random negative example.
- **Precision** measures the proportion of data points predicted as True that are actually True, in other words, the proportion of positive predictions that are true positives.

- **Recall** measures the proportion of data points that are predicted as True, out of all the data points that are actually True. In other words, it measures the proportion of positives that are correctly classified.
- **Accuracy** measures the proportion of data points that are correctly classified.
- **F1-score** is an aggregation of precision and recall.



Reflect on these questions as you complete the executing stage.

- 1. What key insights emerged from your model(s)?
 - 2. What business recommendations do you propose based on the models built?
 - 3. What potential recommendations would you make to your manager/company?
 - 4. Do you think your model could be improved? Why or why not? How?
 - 5. Given what you know about the data and the models you were using, what other questions could you address for the team?
 - 6. What resources do you find yourself using as you complete this stage? (Make sure to include the links.)
 - 7. Do you have any ethical considerations in this stage?
-
- 1. The Decision Tree model achieved good scores, meaning it is a good model that can adapt well to new data while still providing good results. The variables 'satisfaction_level', 'last_evaluation', 'average_monthly_hours', and 'number_project' seem to be very important when it comes to the decision for employees to leave.
 - 2. Regarding the group of overworked employees have very low satisfaction level. Their monthly hours and number of projects should be heavily reduced, in order to stay closer to the average. This should improve their satisfaction level, and thus, their tendency to stay at the company. Regarding the group of underworked employees, most of them may choose to leave due to seeking a full time job, or due to not getting enough projects per month. Their work hours should also be balanced in order to stay closer to the average.
 - 3. A system should be implemented to evaluate when employees are at risk of leaving and adjust their workload to make sure that won't be a factor that can lead to them leaving. Promotions should also be awarded to employees who are performing above average, seeing as the vast majority of the employees who left were not promoted. The models developed in

this analysis and the data collection should also undergo constant improvement.

- 4. There is margin for improvement in the model. Some of the data collected could be more tailored towards model construction, such as the salary variable having the actual salary of the employees or a numerical range. Other factors that may affect the decision of employees leaving, such as employees' relationship with coworkers and management could also be measured and added to the data. A feedback loop can also be established in order to incorporate new data and evolving business needs into the model. This would ensure continuous relevance and accuracy.
- 5. I would like to know how does the career development plan look like for most employees, considering the very low promotion rate, and how/why there is such a big discrepancy between employees' work hours and number of projects per month.
- 6. The documentation of the Scikit-learn package (<https://scikit-learn.org>), and the Stack Overflow website (<https://stackoverflow.com/>) for troubleshooting.
- 7. Decision trees can be particularly susceptible to overfitting. The model might get extremely good at predicting seen data, but as soon as new data is introduced, it may not work nearly as well. Whenever new data is introduced to the model, it should be done so very carefully. Additionally, despite the fact that Decision Trees have low bias, it's still a major factor to be considered.

Step 4. Results and Evaluation

- Interpret model
- Evaluate model performance using metrics
- Prepare results, visualizations, and actionable steps to share with stakeholders

Summary of model results

- Logistic Regression

The logistic regression model achieved a precision of 79%, recall of 82%, f1-score of 80% (all weighted averages), and accuracy of 83%.

- Tree-based Machine Learning

After conducting feature engineering, the decision tree model achieved AUC of 95%, precision of 85%, recall of 90%, f1-score of 87%, and accuracy of 95%, on the test set.

Conclusion, Recommendations, Next Steps

Most of the employees are leaving due to their average monthly work hours either being too high or too low. A system should be implemented to make sure employees are working a minimum and a maximum amount of monthly hours close to the average of 166 hours.

The number of hours per month has a very high correlation with the amount of projects they work on, so they should also be tailored.

With an AUC score of 0.95 after conducting feature engineering, the Decision Tree model performs very well, and can be used appropriately with more data. Caution should still be exercised when introducing new data due to bias.

This notebook was converted with convert.ploomber.io